

二次予選 解説

2022年12月13日・情報オリンピック日本委員会

第22回日本情報オリンピック 二次予選

問題 1 年齢の差 (Age Difference)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

解説担当：山縣龍人

この問題は、各 $i = 1, 2, \dots, N$ に対し、 $|A_{\{1\}} - A_{\{i\}}|$, $|A_{\{2\}} - A_{\{i\}}|$, ..., $|A_{\{N\}} - A_{\{i\}}|$ のうちの最大値を出力する問題である。

小課題 1

小課題 1 では $N = 2$ であった。したがって、 $|A_{\{1\}} - A_{\{2\}}|$ を 2 回出力すれば良い。

小課題 2

各 $i = 1, 2, \dots, N$ に対し、 $|A_{\{1\}} - A_{\{i\}}|$, $|A_{\{2\}} - A_{\{i\}}|$, ..., $|A_{\{N\}} - A_{\{i\}}|$ の値を全て計算し、その最大値を出力することを考える。

小課題 2 では $N \leq 1000$ であるから、時間計算量 $O(N^2)$ のこのアルゴリズムは十分高速である。

小課題 3

小課題 2 では $N \leq 250000$ であるから、 $|A_{\{1\}} - A_{\{i\}}|$, $|A_{\{2\}} - A_{\{i\}}|$, ..., $|A_{\{N\}} - A_{\{i\}}|$ の値を全て計算することは時間制限に間に合わない。

数直線上の $A_{\{1\}}$, $A_{\{2\}}$, ..., $A_{\{N\}}$ の位置に点を打つことを考える。すると、ある人との年齢の差の最大値は、ある点から最も遠い点までの距離と解釈することができる。

点 $A_{\{i\}}$ から最も遠い点は、点 $A_{\{i\}}$ から数直線上の正の方向に最も遠い点か、点 $A_{\{i\}}$ から数直線上の負の方向に最も遠い点のどちらかである。ここで、点 $A_{\{i\}}$ から数直線上の (正 or 負) の方向に最も遠い点は、それぞれ $A_{\{1\}}$, $A_{\{2\}}$, ..., $A_{\{N\}}$ のうちの最大値と最小値に対応する。

したがって、 $mn = \min \{ A_{\{1\}}, A_{\{2\}}, \dots, A_{\{N\}} \}$ 、 $mx = \max \{ A_{\{1\}}, A_{\{2\}}, \dots, A_{\{N\}} \}$ とすれば、 $\max \{ mx - A_{\{i\}}, A_{\{i\}} - mn \}$ が点 $A_{\{i\}}$ から最も遠い点までの距離となる。

以上をまとめると、 $mn = \min \{ A_{\{1\}}, A_{\{2\}}, \dots, A_{\{N\}} \}$ 、 $mx = \max \{ A_{\{1\}}, A_{\{2\}}, \dots, A_{\{N\}} \}$ をあらかじめ計算し、各 $i = 1, 2, \dots, N$ に対して $\max \{ mx - A_{\{i\}}, A_{\{i\}} - mn \}$ を出力すれば良い。

このアルゴリズムは $O(N)$ 時間で動作するから、十分高速である。

原文：<https://www2.ioi-jp.org/joi/2022/2023-yo2/2023-yo2-t1-review.html>

問題 2 ジョイ四人組 (JOI04)

配点 100点 時間制限 3 sec メモリ制限 1024 MB

解説

解説担当：米田優峻

小課題 1 (累計 7 点)

この小課題では $N = 1$ であるため、4 人組を作る方法は「全員を選ぶ」しかない。したがって、「 $A_{\{1\}}, B_{\{1\}}, C_{\{1\}}, D_{\{1\}}$ の最大値」から「 $A_{\{1\}}, B_{\{1\}}, C_{\{1\}}, D_{\{1\}}$ の最小値」を引いた値を出力すれば正解となる。

なお C++ では、最大値は `max` 関数、最小値は `min` 関数を用いて計算することができる（この関数を利用するには `algorithm` をインクルードする必要がある）。たとえば $A_{\{1\}}, B_{\{1\}}, C_{\{1\}}, D_{\{1\}}$ の最大値は、`max({A[1], B[1], C[1], D[1]})` で計算することができる。詳しくは以下のソースコードを参照すること。

```
#include <iostream>
#include <algorithm>
using namespace std;

int N;
int A[75009], B[75009], C[75009], D[75009];

int main() {
    // 入力
    cin >> N;
    for (int i = 1; i <= N; i++) cin >> A[i];
    for (int i = 1; i <= N; i++) cin >> B[i];
    for (int i = 1; i <= N; i++) cin >> C[i];
    for (int i = 1; i <= N; i++) cin >> D[i];

    // 出力
    cout << max({ A[1], B[1], C[1], D[1] }) - min({ A[1], B[1], C[1], D[1] }) << endl;
    return 0;
}
```

小課題 2 (累計 30 点)

まずは重要なキーワードを一つ紹介する。全探索とは、あり得るすべてのパターンを調べることによって答えを求める手法である。たとえばパスワードを忘れてしまったけど携帯にログインしたい場合、0000 から 9999 まで一つずつ調べたくなる人もいるが、これは全探索の一種である。また、家から学校までの最短経路も、（計算に時間はかかるが）家から学校までのすべての経路を調べることで求められる。このように、世の中の多くの問題は、原理的には全探索で解くことができる。

今回の「ジョイ四人組」も、全探索で解くことができる問題の一つである。実際、4 人の選び方を N^4 通り調べ、この中で最大値と最小値の差が一番小さくなるものを出力すれば、正解となる。

さて、このような解法で実行時間制限には間に合うのだろうか。まず、この小課題の制約は $N \leq 30$ であるため、最大ケースの $N = 30$ のとき、全部で $30^4 = 810000$ 通りを調べる必要がある。一方、情報オリンピックのジャッジの計算速度は毎秒数億回程度であるため、実行時間制限の 3 秒には余裕を持って間に合うといえる。（注：家庭用 PC も似たような計算速度になる）

```
#include <iostream>
#include <algorithm>
using namespace std;

int N;
int A[75009], B[75009], C[75009], D[75009];

int main() {
    // 入力
    cin >> N;
    for (int i = 1; i <= N; i++) cin >> A[i];
    for (int i = 1; i <= N; i++) cin >> B[i];
    for (int i = 1; i <= N; i++) cin >> C[i];
    for (int i = 1; i <= N; i++) cin >> D[i];

    // 全探索
    int Answer = 2000000000; // 最初は非常に大きい値にセット
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= N; j++) {
            for (int k = 1; k <= N; k++) {
                for (int l = 1; l <= N; l++) {
                    int Maximum = max({ A[i], B[j], C[k], D[l] }); // 4 人の身長 of 最大値
                    int Minimum = min({ A[i], B[j], C[k], D[l] }); // 4 人の身長 of 最小値
                    Answer = min(Answer, Maximum - Minimum);
                }
            }
        }
    }

    // 出力
    cout << Answer << endl;
    return 0;
}
```

小課題 3（累計 44 点）

この小課題の制約は $N \leq 2000$ と大きいため、残念ながら全探索では間に合わない。実際、 2000^4 は 10 兆を超えるため、絶望的である。だが、ここで特殊な制約「 $A_{\{i\}}, B_{\{j\}}, C_{\{k\}}, D_{\{l\}} \leq 10$ 」を上手く活用することができるか、一度立ち止まって考えてみよう。

まず、同じクラスに同じ身長の人が 2 人以上いても無駄なので、1 人しかいないと考えても答えは変わらない。たとえば、1 年 A 組には生徒が 10 人いて、各生徒の身長が 4, 4, 4, 6, 6, 6, 6, 9, 9 であるとしよう。この場合、1 年 A 組には生徒が 3 人いて、各生徒の身長が 4, 6, 9 であるとみなしても全く問題にならない。これは 1 年 B 組・C 組・D 組でも同じことがいえるため、たとえば

- $N = 10$
- $A = (4, 4, 4, 6, 6, 6, 6, 9, 9)$
- $B = (7, 7, 7, 7, 7, 7, 8, 8, 8)$
- $C = (1, 1, 1, 2, 2, 2, 3, 3, 3)$
- $D = (5, 5, 6, 6, 6, 6, 6, 6, 6)$

というケースの場合、以下のように考えても問題ない。そこで、重複を取り除いたうえで全探索をすると、元々 10^4 通り調べる必要があったところが、 $3 \times 2 \times 4 \times 2 = 48$ 通りしか調べる必要がなくなる。

- $A = (4, 6, 9)$
- $B = (7, 8)$
- $C = (1, 2, 3, 4)$
- $D = (5, 6)$

それでは、この解法で小課題 3 が解けるのかを考えてみよう。制約より、各生徒の身長は 1 以上 10 以下なので、重複を取り除いた後は A, B, C, D の要素数が 10 以下になる。したがって、高々 $10^4 = 10000$ 通りしか全探索をする必要がなく、実行時間制限に間に合うことがわかる。

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int N;
vector<int> A, B, C, D;

int main() {
    // 入力（重複を取り除きやすいうに、vector 型を使っていることに注意！）
    cin >> N;
    A.resize(N, 0); for (int i = 0; i < N; i++) cin >> A[i];
    B.resize(N, 0); for (int i = 0; i < N; i++) cin >> B[i];
    C.resize(N, 0); for (int i = 0; i < N; i++) cin >> C[i];
    D.resize(N, 0); for (int i = 0; i < N; i++) cin >> D[i];

    // A の重複を取り除く
    sort(A.begin(), A.end());
    A.erase(unique(A.begin(), A.end()), A.end());

    // B の重複を取り除く
    sort(B.begin(), B.end());
    B.erase(unique(B.begin(), B.end()), B.end());

    // C の重複を取り除く
    sort(C.begin(), C.end());
    C.erase(unique(C.begin(), C.end()), C.end());

    // D の重複を取り除く
    sort(D.begin(), D.end());
    D.erase(unique(D.begin(), D.end()), D.end());

    // 全探索
    int Answer = 2000000000; // 最初は非常に大きい値にセット
    for (int i = 0; i < (int)A.size(); i++) {
        for (int j = 0; j < (int)B.size(); j++) {
            for (int k = 0; k < (int)C.size(); k++) {
                for (int l = 0; l < (int)D.size(); l++) {
                    int Maximum = max({ A[i], B[j], C[k], D[l] });
                    int Minimum = min({ A[i], B[j], C[k], D[l] });
                    Answer = min(Answer, Maximum - Minimum);
                }
            }
        }
    }
    // Output
    cout << Answer << endl;
    return 0;
}
```

小課題 4 (累計 64 点)

まず、この小課題では身長が 2000 以下と小さいため、身長の最小値を決め打ち全探索することを考える。つまり、次のようなことを考える。

- 身長の最小値が 1 の場合、身長の最大値を最小でいくつにできるか？
- 身長の最小値が 2 の場合、身長の最大値を最小でいくつにできるか？
- 身長の最小値が 3 の場合、身長の最大値を最小でいくつにできるか？
- 身長の最小値が 4, 5, ..., 2000 の場合も同様

それでは、身長の最小値が x のときの答えは、どうやって求めれば良いのだろうか。実は、各クラスから「身長が x 以上である中で最も身長が低い生徒」を選ぶのが最適である。例として、以下のようなテストケースを考えよう。

- $N = 5$
- $A = (152, 154, 166, 170, 200)$
- $B = (156, 162, 168, 183, 191)$
- $C = (145, 160, 173, 175, 186)$
- $D = (185, 187, 193, 205, 230)$

たとえば「身長の最小値を 160 にする」と決めた場合、1 年 A 組からは 160 以上で身長が一番低い 166 センチを選ぶのが最適である。同様に、B 組からは 162 センチ、C 組からは 160 センチ、D 組からは 185 センチを選ぶのが最適である。そのため、身長の最大値として考えられる最小の値は、 $\max(166, 162, 160, 185) = 185$ となる。

そして、160 以外の場合についても頑張って調べると、出力すべき答えが 14 であることが分かる。（注：身長の最小値が 186 のときが最適である。このとき、身長の最大値として考えられる最小の値は、 $\max(200, 191, 186, 187) = 200$ となる）

それでは、この解法の計算回数はどれくらいになるのであろうか。自然に実装すると、身長の最小値が x の場合の答えを求めるのに計算量 $O(N)$ かかる。また、身長は 2000 以下であるため、先程の計算量 $O(N)$ の処理を 2000 回行わなければならない。そのため、大まかな計算回数は $2000 \times N$ となるが、 $N \leq 2000$ であるため、実行時間制限には間に合う。

```

#include <iostream>
#include <algorithm>
using namespace std;

int N;
int A[75009], B[75009], C[75009], D[75009];

// 身長が最小値が LIMIT の場合、最大値として考えられる最小の値を返す
int solve(int LIMIT) {
    int PA = 2000000000; // A 組の LIMIT 以上の最小値
    int PB = 2000000000; // B 組の LIMIT 以上の最小値
    int PC = 2000000000; // C 組の LIMIT 以上の最小値
    int PD = 2000000000; // D 組の LIMIT 以上の最小値
    for (int i = 1; i <= N; i++) {
        if (A[i] >= LIMIT) PA = min(PA, A[i]);
        if (B[i] >= LIMIT) PB = min(PB, B[i]);
        if (C[i] >= LIMIT) PC = min(PC, C[i]);
        if (D[i] >= LIMIT) PD = min(PD, D[i]);
    }

    if (PA == 2000000000 || PB == 2000000000 || PC == 2000000000 || PD == 2000000000) return 2000000000;
    return max({ PA, PB, PC, PD });
}

int main() {
    // 入力
    cin >> N;
    for (int i = 1; i <= N; i++) cin >> A[i];
    for (int i = 1; i <= N; i++) cin >> B[i];
    for (int i = 1; i <= N; i++) cin >> C[i];
    for (int i = 1; i <= N; i++) cin >> D[i];

    // 決め打ち全探索
    int Answer = 2000000000;
    for (int i = 1; i <= 2000; i++) {
        Answer = min(Answer, solve(i) - i);
    }

    // 出力
    cout << Answer << endl;
    return 0;
}

```

小課題 5 (累計 77 点)

この小課題では、身長としてあり得る範囲が 1 以上 10^9 以下と一気に広がる．そのため、小課題 4 の解法では計算回数が $10^9 \times N$ 回必要になってしまい、TLE となってしまう．

しかし、ここで恐れることはない．実は、身長の最小値として 1, 2, ..., 10^9 を全部探索する必要はないのである．具体的には、 $A_{\{i\}}, B_{\{j\}}, C_{\{k\}}, D_{\{l\}}$ に現れる $4N$ 通りを調べれば十分である．たとえば、先程も例に挙げた

- $N = 5$
- $A = (152, 154, 166, 170, 200)$
- $B = (156, 162, 168, 183, 191)$
- $C = (145, 160, 173, 175, 186)$
- $D = (185, 187, 193, 205, 230)$

というケースの場合、「身長の最小値が 150 であるとき、身長の最大値として考えられる最小の値は？」という質問を考えるのは無駄である（身長が 150 の人は誰一人としていないため）．

すると、計算回数を $4N \times N = 4N^2$ 回程度にまで削減することができる。 $N \leq 2000$ であるため、C++ など計算速度の早い言語であれば、AC を得ることができる。

```
#include <iostream>
#include <algorithm>
using namespace std;

int N;
int A[75009], B[75009], C[75009], D[75009];

// 身長が LIMIT の場合、最大値として考えられる最小の値を返す
int solve(int LIMIT) {
    int PA = 2000000000;
    int PB = 2000000000;
    int PC = 2000000000;
    int PD = 2000000000;
    for (int i = 1; i <= N; i++) { if (A[i] >= LIMIT) PA = min(PA, A[i]); }
    for (int i = 1; i <= N; i++) { if (B[i] >= LIMIT) PB = min(PB, B[i]); }
    for (int i = 1; i <= N; i++) { if (C[i] >= LIMIT) PC = min(PC, C[i]); }
    for (int i = 1; i <= N; i++) { if (D[i] >= LIMIT) PD = min(PD, D[i]); }

    if (PA == 2000000000 || PB == 2000000000 || PC == 2000000000 || PD == 2000000000) return 2000000000;
    return max({ PA, PB, PC, PD });
}

int main() {
    // 入力
    cin >> N;
    for (int i = 1; i <= N; i++) cin >> A[i];
    for (int i = 1; i <= N; i++) cin >> B[i];
    for (int i = 1; i <= N; i++) cin >> C[i];
    for (int i = 1; i <= N; i++) cin >> D[i];

    // 決め打ち全探索
    int Answer = 2000000000;
    for (int i = 1; i <= N; i++) Answer = min(Answer, solve(A[i]) - A[i]);
    for (int i = 1; i <= N; i++) Answer = min(Answer, solve(B[i]) - B[i]);
    for (int i = 1; i <= N; i++) Answer = min(Answer, solve(C[i]) - C[i]);
    for (int i = 1; i <= N; i++) Answer = min(Answer, solve(D[i]) - D[i]);

    // 出力
    cout << Answer << endl;
    return 0;
}
```

小課題 6 (累計 100 点)

まず小課題 5 の解法では、「身長が x 以上の中の最小値」を各クラスについて求める必要があった。そして小課題 5 では、これを全探索によって $O(N)$ かけて求めていた（上の実装例の `solve` 関数を参照）。

しかし、配列 `A`, `B`, `C`, `D` を予めソートしておく、配列の二分探索によってこの値を $O(\log N)$ で求めることができる（知らない方はインターネット等で調べてみることを推奨する）。そのため、プログラム全体の計算量が $O(N \log N)$ に改善され、ついに満点を得ることができる。（実装例はサンプルソース (C++, Python) を参照。）

原文： <https://www2.ioi-jp.org/joi/2022/2023-yo2/2023-yo2-t2-review.html>

問題 3 塗りつぶし (Painting)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

解説担当：菅井遼明

小課題 1

小課題 1 では $H=1$ が成り立っています。このとき、各マスの領域は必ず縦の長さが 1 の長方形になります。

マス目を隣接するマスの色が異なる部分で区切り、いくつかの長方形に分割します。それぞれの長方形について、その中のマスを指定したときの領域は以下のようになります。

- 左右の長方形の色が異なる場合、左隣の長方形の色を指定することで、自身と左隣の長方形が領域となる。
- 左右の長方形の色が異なる場合、右隣の長方形の色を指定することで、自身と右隣の長方形が領域となる。
- 左右の長方形の色が同じ場合、その色を指定することで、自身と両隣の長方形が領域となる。

適切に場合分けを行うことで、時間計算量 $O(W)$ で解くことができますが、それより悪い計算量であっても得点を得ることができます。

小課題 2

塗りつぶしを使うときに指定する色は、1, 2, 3, 4, 5 のみ考えればよいです。よって、指定するマスと色の組み合わせを全探索し、それぞれについて得点を計算することで解くことができます。

マスを指定したとき、そのマスの領域に含まれるマスを列挙する必要があります。これを行うには、以下の方法があります。

- 幅優先探索 (BFS) を行う。
- 深さ優先探索 (DFS) を行う。
- 素集合データ構造 (Union-Find 木, DSU) を用いる。

時間計算量 $O(HW)$ など で解けます。

小課題 3

小課題 2 と比べて色の種類が増えていますが、実は考える必要のある色の個数はかなり絞り込むことができます。

具体的には、マス (x,y) を指定して塗りつぶしを使うとき、マス (x,y) の領域が大きくなる色のみ考えればよいです。よって、マス (x,y) の領域に隣接するマスの色が候補となります。

さらに、マス (x,y) に直接隣接するマスの色しか考えなくてよいことがわかります。なぜなら、マス (x,y) の領域に含まれるマスであれば、同じ色を指定したときの結果は同じになるので、離れたマスに隣接する色はそのマスを指定したときに調べることにできるからです。

上の考察により、指定するマスを定めたときの指定する色の候補は最大 4 つとなるので、適切に実装すればこれらの候補をすべて試しても制限時間に間に合います。

時間計算量は $O(H^2 W^2)$ となります。

小課題 4

色が異なる隣接するマスの境界で長方形を区切ると、長方形は何個かの部分に分かれます。これらの部分を連結成分と呼ぶことにします。また、連結成分に含まれるマスの個数をその連結成分の大きさと呼ぶことにします。

2 つのマス A と B が同じ連結成分に含まれるとき、塗りつぶしを使う際に A を指定しても B を指定しても同じ結果になります。よって、塗りつぶしの操作は、ある連結成分と色を選び、その連結成分に含まれるマスの色をすべてその色にする、と言い換えられます。

小課題 3 では、色は 1, 2 の 2 種類しかないので、塗りつぶしをするときはその連結成分の色と異なる色を指定する場合のみ考えればよいです。このとき、指定した連結成分はその連結成分と辺で接するすべての連結成分と合体します。よって、JOI 君の得点は、指定した連結成分の大きさに、隣接する連結成分の大きさをすべて足したものになります。

連結成分に番号を振り、各マスがどの連結成分に含まれるかをあらかじめ計算します。これは幅優先探索、深さ優先探索、素集合データ構造などで行うことができます。次に、隣接する連結成分の組を列挙します。境界の両側のマスをすべて調べることで容易に実装できます。最後に、指定する連結成分をすべて試し、それぞれについて得点を計算します。

時間計算量は $O(HW)$ となります。

小課題 5

小課題 4 と同様に、各マスが含まれる連結成分と連結成分同士の隣接関係を調べておきます。

ある連結成分を色 c で塗りつぶしたとき、その連結成分の大きさは、隣接する色 c の連結成分の大きさの和だけ増加します。よって、この増加量が最大になる色を選ぶのが最適です。

色を表す整数が大きいので、単純にそれぞれの色について合計を管理することはできません。連想配列を用いたり、小さい数になるように色の番号を付け替えたりするなどの工夫をすると、効率の良いアルゴリズムが得られ、正解することができます。

時間計算量は解法によりませんが、典型的な実装では $O(HW \log HW)$ となります。

原文 : <https://www2.ioi-jp.org/joi/2022/2023-yo2/2023-yo2-t3-review.html>

問題 4 貨物列車 (Freight Train)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

解説担当：星井智仁

はじめに

この問題は実行時間制限がやや厳しいため、Python などの実行速度が遅い言語ではこの解説のアルゴリズムをそのまま実装しても満点が得られないかもしれません。

小課題 1

小課題 1 では、駅に置かれている貨物の価値 $A_{\{i\}}$ がすべて 1 であり、かつ列車に詰める荷物の最大値 W も 1 です。

このとき、 $A_{\{i\}}$ がすべて等しいので、駅 1 に近い駅の貨物を優先して輸送するのが明らかに最適です。

駅 i の貨物を駅 1 に輸送するためには、往復するのに距離 $(i-1) \times 2$ だけ列車が走行する必要があります。

駅 2, 3, ..., k の貨物を駅 1 に輸送するために列車が走行する距離は、 $\sum_{i=2, \dots, k} \{ (i-1) \times 2 \} = (2-1) \times 2 + (3-1) \times 2 + \dots + (k-1) \times 2$ です。この値が D 以下となる最大の整数 k を求めると、 $k-1$ が答えとなります。（制約より、 $\sum_{i=2, \dots, N} \{ (i-1) \times 2 \} = N^2 - N \geq D$ なので k の上限 N を気にする必要はありません。）

方程式を解いて計算量 $O(1)$ で解いたり、二分探索を用いて計算量 $O(\log N)$ で解くこともできますが、愚直にループを回して計算量 $O(N)$ で解いても実行時間制限には問題なく間に合うでしょう。

小課題 2

小課題 2 では、小課題 1 と同様に貨物の価値 $A_{\{i\}}$ がすべて 1 ですが、 W に特別な制約がかかっているわけではありません。

$A_{\{i\}}$ がすべて等しいので、駅 1 に近い駅の貨物を優先して輸送するのが最適ですが、複数の貨物を列車に積むことができるため、どのように列車を走行させるのが最適か考察する必要があります。

駅 2, 3, ..., k の貨物を駅 1 に輸送することを考えましょう。このとき、以下のように列車を運行させるのが最適です。

- 列車に駅 1, 駅 k 間を往復させる。
 - 駅 k , $k-1$, ..., $k-W+1$ に置いてある貨物を積み、駅 1 に輸送する。

- 列車に駅 1, 駅 $k-W$ 間を往復させる。
 - 駅 $k-W$, $k-W-1$, ..., $k-2W+1$ に置いてある貨物を積み, 駅 1 に輸送する。
- 列車に駅 1, 駅 $k-2W$ 間を往復させる。

:

- 列車に駅 1, 駅 $k-nW$ 間を往復させる。
 - 駅 $k-nW$, $k-nW-1$, ..., $k-(n+1)W+1$ に置いてある貨物を積み, 駅 1 に輸送する。
- 列車に駅 1, 駅 $k-(n+1)W$ 間を往復させる。
 - 駅 $k-(n+1)W$, $k-(n+1)W-1$, ..., 3, 2 に置いてある貨物を積み, 駅 1 に輸送する。

以上のような輸送方法ができる k の最大値を調べればよいです。

これを愚直に実装すると計算量は $O(N^2)$ でしょうか。もっと工夫して計算量を削減することはできますが、計算量 $O(N^2)$ でも十分実行時間制限には間に合うでしょう。

小課題 4

解説の都合上, 先に小課題 4 について説明します。

小課題 4 では, 駅の数 N が 15 以下です。 N が小さいため, bit 全探索という手法を用いることができます。

bit 全探索とは, ある集合の大きさを M としたとき, その集合の部分集合 $2^{\{M\}}$ 通りすべてについて調べ上げるという手法です。

どの駅の貨物を駅 1 に輸送するか, というのは $2^{\{N-1\}}$ 通りあります。この $2^{\{N-1\}}$ 通りの中の, 輸送可能な選び方のうち貨物の価値の合計が最大となるものが答えです。

輸送可能か否かは, 小課題 2 と同様の方法で距離を計算することができます。

例えば, $W=2$ で, 駅 2, 3, 5, 7, 11 の貨物を選んだとき, 以下のように列車を運行させるのが最適です。

- 列車に駅 1, 駅 11 間を往復させる。
 - 駅 11, 駅 7 に置いてある貨物を積み, 駅 1 に輸送する。
- 列車に駅 1, 駅 5 間を往復させる。
 - 駅 5, 駅 3 に置いてある貨物を積み, 駅 1 に輸送する。
- 列車に駅 1, 駅 2 間を往復させる。
 - 駅 2 に置いてある貨物を積み, 駅 1 に輸送する。

なお, 小課題 1, 2 と異なり, 各駅に置かれている貨物の価値 A がすべて同じ値というわけではありませんが, 途中駅で貨物を降ろすことを考える必要はありません。

なぜなら, 選んだ貨物は必ず駅 1 まで輸送しなければならず, 選ばなかった貨物はそもそも列車に積む必要がありません。なので, 貨物を途中駅で降ろしたり, 積み替えたりするメリットはありません。

この方法により, 計算量 $O(N \times 2^{\{N\}})$ で解くことができます。

小課題 3

小課題 3 では、列車に積める貨物の数の最大値 W が 1 です。小課題 4 の説明で既に述べたように、途中駅で貨物を降ろすメリットはありません。そのため、各駅に置かれている貨物について独立に考えることができます。

- 駅 2 に置かれている貨物を駅 1 に輸送すると距離 2 走行する必要があるが、価値 $A_{\{2\}}$ が得られる。
- 駅 3 に置かれている貨物を駅 1 に輸送すると距離 4 走行する必要があるが、価値 $A_{\{3\}}$ が得られる。
- $\vdots$
- 駅 N に置かれている貨物を駅 1 に輸送すると距離 $(N-1) \times 2$ 走行する必要があるが、価値 $A_{\{N\}}$ が得られる。

これはナップサック問題に帰着できます。ナップサック問題とは、動的計画法 という手法で解くことができることが知られている、非常に有名な問題です。

ナップサック問題の形に書き換えると、以下のような問題になります。（ナップサック問題にはいくつかバリエーションがあります。ここでは、同じ種類の品物を 2 つ以上選ぶことを許していません。許すものもあります。）

- 重さ D 以下まで品物を入れられるナップサックがあります。品物 i ($2 \leq i \leq N$) の価値は $A_{\{i\}}$ ，重さは $(i-1) \times 2$ です。ナップサックに品物をいくつか選んで入れるとき、ナップサックに入れることができる品物の価値の合計の最大値はいくつでしょうか？

これは以下のように動的計画法を用いて解くことができます。

二次元配列を用意します。ここでは、この配列の名前を dp とします。

$dp[i][j] :=$ 品物 2, 3, ..., i について、それぞれナップサックに入れるか否かが決め、その重さの合計が j であるときのナップサックに入れられる価値の最大値（存在しない時は $-\infty$ ）

と定義します。

このとき、漸化式は $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-(i-1) \times 2] + A_{\{i\}})$ です。

初期値として $dp[1][0]=0$ ， $dp[k][l]=-\infty$ ($(k,l) \neq (1,0)$) を設定し， i ， j が小さい方から順々にこの漸化式を計算して $dp[i][j]$ を求めていくと二次元配列 dp の各値が求まります。

そうして求めた $dp[N][0]$ ， $dp[N][1]$ ， $\dots$ ， $dp[N][D]$ の最大値が答えとなります。計算量は $O(ND)$ です。 $D \leq N^2 - N$ なので、計算量は $O(N^3)$ です。

小課題 5

制約は駅の数 $N \leq 50$ のみです。 W に特別な制約はかかっていません。

小課題 3, 4 と同様に、いくつかの駅に置いてある貨物を選び、選んだ貨物のみを駅 1 に運ぶことを考えます。

小課題 3 では品物 2（駅 2 の貨物），品物 3（駅 3 の貨物）， $\dots$ ，品物 N （駅 N の貨物）と、駅 1 に近い方から順番にそれぞれの品物（貨物）を選ぶか選ばないか決めていましたが、逆に、駅 N の貨物，駅 $N-1$ の貨

物, ..., 駅 2 の貨物という順番で選ぶか選ばないかを考えることにします。動的計画法に落とし込めないか考えると, 以下のような漸化式が立ちます。

$dp[i][j][k] :=$ 駅 i , 駅 $i+1$, ..., 駅 N の貨物を選ぶか選ばないか決めたとき, ここまでに選んだ貨物の数が j であり, 列車の総走行距離が k であるときの貨物の価値の合計の最大値

と定義します。

漸化式は以下のようになります。

- $j \equiv 1 \pmod{W}$ のとき: $dp[i][j][k] = \max (dp[i+1][j][k], dp[i+1][j-1][k-(i-1) \times 2] + A_{\{i\}})$
- そうでないとき: $dp[i][j][k] = \max (dp[i+1][j][k], dp[i+1][j-1][k] + A_{\{i\}})$

初期値として, $dp[N+1][0][0]=0$, $dp[l][m][n]=-\infty$ ($(l,m,n) \neq (N+1,0,0)$) を設定し, i が大きい方, j , k が小さい方からこの漸化式を順々に計算して $dp[i][j][k]$ を求めていくと三次元配列 dp の各値が求まります。

そうして求めた $dp[2][0][0]$, $dp[2][0][1]$, ..., $dp[2][0][N-1]$, $dp[2][1][0]$, ..., $dp[2][1][N-1]$, ..., $dp[2][D][0]$, ..., $dp[2][D][N-1]$ の最大値が答えとなります。計算量は $O(N^2 D)$ です。 $D \leq N^2 - N$ なので, 計算量は $O(N^4)$ です。

小課題 6

小課題 5 の解法を改善することで満点を得ることができます。

小課題 5 では

$dp[i][j][k] :=$ 駅 i , 駅 $i+1$, ..., 駅 N の貨物を選ぶか選ばないか決めたとき, ここまでに選んだ貨物の数が j であり, 列車の総走行距離が k であるときの貨物の価値の合計の最大値

として漸化式を立てました。

まず, j の値の範囲を狭めることができます。「ここまでに選んだ貨物の数」ではなく「ここまでに選んだ貨物の数を W で割った余り」としても問題ありません。漸化式は以下のようになります。

ここで, $t = W-1$ ($j = 0$ のとき), $j-1$ ($j \neq 0$ のとき) とします。

- $j = 1$ のとき (または $W = 1$ のとき):
 $dp[i][j][k] = \max (dp[i+1][j][k], dp[i+1][t][k-(i-1) \times 2] + A_{\{i\}})$
- $j \neq 1$ のとき (かつ $W \neq 1$ のとき): $dp[i][j][k] = \max (dp[i+1][j][k], dp[i+1][t][k] + A_{\{i\}})$

このときの計算量は $O(NWD)$ です。一見改善されているように見えますが, $W \leq N-1$, $D \leq N^2 - N$ なので計算量は $O(N^4)$ のまま変わりません。

しかし, よく考えてみましょう。 D は総走行距離の上限として与えられていますが, すべての貨物を駅 1 に輸送するために走行しなければならない総距離の最小値は本当に $N^2 - N$ なのでしょうか?

$W=1$ のとき, 小課題 1 の説明に書いたように, 上限は $\sum_{i=2, \dots, N} \{ (i-1) \times 2 \} = N^2 - N$ です。

しかし、 $W=N-1$ のときはどうでしょうか。列車は駅 1 と駅 N を 1 往復するだけですべての貨物を駅 1 に輸送することができます。つまり、 $W=N-1$ のときは $(N-1) \times 2$ が実質的な D の上限なのです。

W が大きくなればなるほど実質的な D の上限は小さくなります。具体的には、

D の実質的な上限 $D_{\max} = (N-1) \times 2 + (N-W-1) \times 2 + (N-2W-1) \times 2 + \dots \leq N^2/W$ となります。（これは雑な見積もりなので上限値を N^2/W としてプログラムを提出すると不正解になります。プログラム上で $(N-1) \times 2 + (N-W-1) \times 2 + (N-2W-1) \times 2 + \dots$ を計算しましょう。）

D の値が $D_{\max}$ 以上のとき、 $A_2 + A_3 + \dots + A_N$ を出力、そうでないとき動的計画法を行う、とすると、計算量は $O(NWD_{\max})$ です。 $O(D_{\max}) = O(N^2/W)$ なので、 $O(NWD_{\max}) = O(N^3)$ となり満点を得ることができます。

原文：<https://www2.ioi-jp.org/joi/2022/2023-yo2/2023-yo2-t4-review.html>

問題 5 日本沈没 2 (Japan Sinks 2)

配点 100点

時間制限 3 sec

メモリ制限 1024 MB

解説

解説担当：松尾凜太郎

小課題 1

西風の嵐では西から順に、東風の嵐では東から順に、強さと同じ数だけの区画を見ていきます。「今見ている区画より左の区画の標高の最大値」を持ちながら今見ている区画の標高が減るかを判定していけば、全体で時間計算量 $O(NQ)$ で処理することができます。

小課題 2

嵐はすべて強さ N であるという制約の小課題です。

i 日目の出来事の直前に、「それより西のどの区画よりも(真に)標高が高い区画」の番号の集まりを $L_{\{i\}}$ ，「それより東のどの区画よりも(真に)標高が高い区画」の番号の集まりを $r_{\{i\}}$ で表すことにします。たとえば5日目の出来事の直前に各区画の高さが西から順に 3, 2, 4, 4, 1, 3 だったとすると $L_{\{i\}} = \{1, 3\}$, $r_{\{i\}} = \{4, 6\}$ です。また、 $L_{\{i\}}$ と $r_{\{i\}}$ の要素を合わせたものを $t_{\{i\}}$ とします。すると、この小課題に限らず一般に次のことが成り立ちます。

- $L_{\{i\}}$ のどの要素も、 $r_{\{i\}}$ のどの要素よりも小さい。
- i 日目の直前において、区画 j の標高について次のことが成立する。

$t_{\{i\}}$ に含まれる要素のうち、 j 以下で j に一番近いものを L ， j 以上で j に一番近いものを R とする。

i 日目の直前の区画 j の標高は、その時点での区画 L と区画 R の標高、及び初期状態の区画 j の標高の3つの値の最小値である。

また、強さ N の嵐しか起こらないこの小課題では以下のことが成り立つことが分かります。

- $L_{\{i+1\}}$ は、 $L_{\{i\}}$ のうち大きい方からいくつか (0 個の可能性もある) を取り除いてできる。
- $r_{\{i+1\}}$ は $r_{\{i\}}$ のうち小さい方からいくつか (0 個の可能性もある) を取り除いてできる。

最後の項目については、嵐が起きた時の各地点の標高の変化を考えると帰納的に示せます。

従って、嵐に応じて $L_{\{i\}}$ の大きい方、 $r_{\{i\}}$ の小さい方からそれぞれ適切に要素を削除し、今 $L_{\{i\}}$, $r_{\{i\}}$ に入っている番号の区画の標高が開始から何 m 減ったかを記録していくとよいです。標高を報告する際には

$l_{\{i\}}$, $r_{\{i\}}$ の中で二分探索をして L, R を出すことで、求める標高を計算することができます。計算量は $O((N + Q) \log(N))$ となります。

小課題 3

この小課題では、以下のことが成り立ちます。

- 任意の時点で、標高が最大値をとる地点の番号は 1 つの区間で表すことができる。 i 日目の直前において、標高が最大値をとる地点の番号が $u_{\{i\}}$ 以上 $v_{\{i\}}$ 以下であるとする。 ($u_{\{i\}} \leq v_{\{i\}}$)
- i 日目に強さ x の西風の嵐が起こる時: 西から数えて $\min(x, v_{\{i\}})$ 個の区画の標高が 1 減り、それ以外の区画の標高は変わらない。
 - $x < v_{\{i\}}$ のとき, $l_{\{i+1\}} = \max(u_{\{i\}}, x+1)$, $r_{\{i+1\}} = v_{\{i\}}$ となる。
 - $x \geq v_{\{i\}}$ のとき, $l_{\{i+1\}} = u_{\{i\}}$, $r_{\{i+1\}}$ は、この嵐の直後の時点の区画 $v_{\{i\}}$ から東に区画 $v_{\{i\}}$ と等しい標高が連続する一番東の区画の番号となる。
- 東風についても、西風の場合と左右対称で同様のことが起こる。

従って、区間加算と区間最小値取得の機能を持つ遅延伝播セグメント木と呼ばれるデータ構造で各区画の標高を数列として管理すればよいです。

「区画 $v_{\{i\}}$ から東に区画 $v_{\{i\}}$ と等しい標高が連続する一番東の区画」については、数列を a として、 a の $v_{\{i\}}, v_{\{i\}} + 1, \dots, j$ 番目の最小値が $a_{\{v_{\{i\}}\}}$ と等しくなる最大の j を求めればよいので、二分探索をすればよいです。

単純に二分探索して $O(\log(N))$ 回セグメント木に対して最小値クエリを行うと時間計算量は全体で $O(N + Q (\log(N))^2)$ となります。また、セグメント木の内部構造に立ち入って二分探索を行うと全体で $O(N + Q \log(N))$ とすることもできます。

小課題 4

この小課題の解法は本質的に小課題 5 の解法と同じ (2 種類の嵐のうち片方だけ処理するとこの小課題の解法になる) なので、小課題 5 の解法を参照してください。

小課題5 (満点)

小課題 2 の項で定義した $l_{\{i\}}, r_{\{i\}}$ を管理していく方針をとります。小課題 2 の項で「一般に次のことが成り立ちます。」とある項目の内容はこの小課題でも使うことができます。 i 日目の強さ x の西風の嵐のときを考えます。 $l_{\{i\}}$ の要素で小さい方から j 番目のものを $l_{\{i,j\}}$ で表すこととします。

- x が $r_{\{i\}}$ の最小値以上の場合: 小課題 2 と同様に r が変化する。 l は, $l_{\{i+1\}} = l_{\{i\}}$ で変化しない。
- x が $r_{\{i\}}$ の最小値未満の場合:

$l_{\{i\}}$ の要素のうち, x 以下で x に一番近いものを L とする。 $l_{\{i\}}$ の要素のうち, x より大きく x に一番近いものを R とする。そのようなものが存在しなければ $r_{\{i\}}$ の最小値を R とする。

もし $x < j < R$ となる j であって、嵐の直前に区画 j と区画 L の標高が等しかったような j が存在すれば、そのうち最小のものは新たに $L_{\{i+1\}}$ に入る。 $L_{\{i+1\}}$ からは R が削除されるかもしれない。このような区画 j は今までに 1 度も標高が変化していないため、元の数列 A について「 x 番目より後で初めて y 以上となるのは何番目か」という質問に答えることによって求めることができる。

東風の嵐の場合も左右対称の処理をすればよいです。元の数列 A についての「 x 番目より後で初めて y 以上となるのは何番目か」という質問については、最大値取得の機能を持ったセグメント木上での二分探索や、最大値を保持したダブリングと呼ばれている手法により $O(\log(N))$ の計算量で答えることができます。 $L_{\{i\}}$, $r_{\{i\}}$ は例えば C++ における `std::set` を用いて管理すると $O(\log(N))$ で新たな要素の追加、二分探索ができます。また、 $L_{\{i\}}$, $r_{\{i\}}$ に含まれる区画のうち西から、または東からいくつかの区画に 1 を足す処理は遅延セグメント木を使うことで $O(\log(N))$ で実現できます。(差分を管理すれば一点加算区間和取得のセグメント木や Binary Indexed Tree で済みます。) 以上より、この問題は $O((N + Q) \log(N))$ で解くことができます。

原文：<https://www2.ioi-jp.org/joi/2022/2023-yo2/2023-yo2-t5-review.html>

出典：情報オリンピック日本委員会「第22回日本情報オリンピック JOI 2022/2023 二次予選競技課題」。原資料は CC BY-SA 4.0 で提供されています。
本PDFは各解説ページを問題順に再構成したものです。
<https://www2.ioi-jp.org/joi/2022/2023-yo2/index.html>